Hierarchical Reinforcement Learning for Air Combat at DARPA's AlphaDogfight Trials

Adrian P. Pope*, Jaime S. Ide*, Daria Mićović, Henry Diaz, Jason C. Twedt, Kevin Alcedo, Thayne T. Walker, David Rosenbluth, Lee Ritholtz, and Daniel Javorsek II

arXiv:2105.00990v3 [cs.LG] 20 Jul 2026

***Abstract*—Autonomous control in high-dimensional, continuous state spaces is a persistent and important challenge in the fields of robotics and artificial intelligence. Because of high risk and complexity, the adoption of AI for autonomous combat systems has been a long-standing difficulty. In order to address these issues, DARPA's AlphaDogfight Trials (ADT) program sought to vet the feasibility of and increase trust in AI for autonomously piloting an F-16 in simulated air-to-air combat. Our submission to ADT solves the high-dimensional, continuous control problem using a novel hierarchical deep reinforcement learning approach consisting of a high-level policy selector and a set of separately trained low-level policies specialized for excelling in specific regions of the state space. Both levels of the hierarchy are trained using off-policy, maximum entropy methods with expert knowledge integrated through reward shaping. Our approach outperformed human expert pilots and achieved a second-place rank in the ADT championship event.**

***Impact Statement*—Significant performance milestones in reinforcement learning have been achieved in recent years, with autonomous agents demonstrating super-human performance across a wide variety of tasks. Before these algorithms can be extensively deployed in real-world defense applications, a greater level of trust must first be achieved. ADT was an important step towards developing the trust necessary to operationalize these algorithms, by demonstrating their effectiveness on a foundational yet relevant problem in a high-fidelity simulation environment. Developed for the program, our hierarchical reinforcement learning agent was designed alongside of and competed against active fighter pilots, and ultimately defeated a graduate of the United States Air Force's F-16 Weapons Instructor Course in match play.**

***Index Terms*—Hierarchical Reinforcement Learning, Deep Reinforcement Learning, Artificial intelligence, Autonomy, Air Combat.**

* These authors contributed equally to this work.

Adrian P. Pope and Lee Ritholtz are with Primordial Labs, and at the time of the work were with the Lockheed Martin Artificial Intelligence Center, Applied AI Team. (e-mail: adrian@primordial-labs.com).

Jaime S. Ide, Daria Mićović, Henry Diaz, David Rosenbluth, Jason C. Twedt, Thayne T. Walker, and Kevin Alcedo are with the Lockheed Martin Artificial Intelligence Center, Applied AI Team. (e-mail: jaime.s.ide@lmco.com).

Daniel Javorsek II is with the United States Air Force.

This article is based upon work sponsored by the Defense Advanced Research Projects Agency (DARPA).

This work has been accepted for publication in *IEEE Transactions on Artificial Intelligence*. Published version: vol. 4, no. 6, pp. 1371–1385, 2023, doi: 10.1109/TAI.2022.3222143.

© 2022 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.

I. Introduction

THE Defense Advanced Research Projects Agency's (DARPA) Air Combat Evolution (ACE) program, seeks to build and enhance trust in autonomy for air-to-air combat and ultimately culminated in a live flight exercise with full-scale aircraft. The AlphaDogfight Trials (ADT) were created as a precursor to the larger ACE program in order to minimize risk. For ADT, teams proposed algorithmic solutions for one-vs-one within visual range (WVR) air combat, which is also known as a *dogfight*. Solutions submitted to ADT ranged from rule-based systems to end-to-end machine learning, and eight finalists were selected to perform on the program. Each team's solution was tested in simulated dogfights in a high-fidelity simulation environment that leveraged an F-16 flight dynamics model (FDM). Scrimmages were held against a variety of opponents including DARPA-provided agents exhibiting various behaviors (e.g., cruise missile-like behavior), other finalist's agents, and expert human fighter pilots.

In deployment, autonomous flight control software such as autopilots and terrain avoidance systems are currently limited to deterministic and rule-based approaches, which can be attributed to a general lack of trust in AI-based autonomy. Before any pilot can advance to more complicated missions such as suppression of air defenses, escorting, and point protection, they must first master the basic flight maneuvers required for WVR air combat. For this reason, ACE selected the dogfight as the starting point for developing trust in autonomous air-combat systems.

In this article, we describe the simulation environment, the design and training of our agent, and the evaluation results showing the evolution of the agent ultimately submitted to the competition. Our approach uses hierarchical reinforcement learning (RL) that dynamically selects from a set of policies which are specialized for various aspects of an engagement. Our agent defeated a graduate of the United States Air Force (USAF) F-16 Weapons Instructor Course in match play with a perfect record (5 W - 0 L) and achieved 2^{nd} place overall in the final tournament.

The main contributions of this work are as follows:

- Successful application of RL to train an agent capable of dogfighting in a high-fidelity simulation environment, at a level that exceeds human experts. In particular, we develop a hierarchical deep RL agent boosted by a variety of modern techniques including soft actor-critic, curriculum learning, distributed training, priority

experience replay, and n-step returns.
- Detailed descriptions of reward shaping optimized for the dogfight scenario (Sections V-B).
- Proposal of a novel hierarchical mechanism using a high-level policy selector to optimally switch between previously trained low-level policies given the current context of the engagement (V-C). We provide a set of analyses demonstrating the effectiveness of this approach (VII-B).
- Presentation of training and evaluation frameworks along with strategies to develop a high-performance autonomous agent (Section VI).

II. Related Work

Research on algorithms for autonomous air combat has been active since the 1950s [1]. Some approaches sought to use expert knowledge to formulate counter maneuvers as rule-based methods for a variety of positional contexts [2] [3]. Other efforts have codified air-to-air scenarios as optimization problems to be solved computationally [3]–[7].

Some research relies on building utility functions over a discrete set of actions and utilizing game theory [6], [7]. Other approaches employ various forms of dynamic programming (DP) [4], [5], [8]. In many of these approaches, trade-offs are made in the complexity of the algorithm and environment representation to reach approximately optimal solutions within a reasonable time frame [4]–[8]. One notable work used Genetic Fuzzy Trees to develop an agent that defeated a USAF weapon school graduate in the USAF's Advanced Framework for Simulation, Integration and Modeling (AFSIM) [9].

More recently, deep RL has been applied to this problem [10]–[18]. For example, Yang *et al.* [13] leveraged deep-q networks to train an agent that selects from a collection of discrete maneuvers in a custom 3-D environment. Zhang *et al.* [10] evaluated a variety of RL algorithms applied to air missions modeled in AFSIM and found agents capable of learning cooperative strategies. Yoo *et al.* leveraged recurrent neural networks to effectively predict opponent trajectories [17]. Sun *et al.* developed a multi-agent RL algorithm with a hierarchical decision network that allows for the discrete selection of basic maneuvers (e.g., climb, turn, descend, etc.) and continuous actions to discipline those maneuvers [18].

Most of the deep RL approaches surveyed either abstract the action space to high-level behaviors or rely on low-fidelity simulation environments. The former approach limits the agent's expressiveness and potentially its effectiveness, and the latter approach often bears little resemblance to real-world applications [10]–[15]. The simulation environment used in this work, JSBSim, is uniquely high-fidelity in comparison to those used in related works. It provides an FDM of an F-16 aircraft with six degrees of freedom that accepts inputs to the flight control system. JSBSim is open-source software that is generally considered to be very accurate for modeling aerodynamics and the physics of flight [19], [20]. In this article, we outline the design of an RL agent with hierarchical, continuous control of the aircraft that is capable of executing highly competitive tactics within this high-fidelity environment.

Independent of autonomous air combat research, there has been significant research on the topic of hierarchical RL [21], [22]. Dividing a complex task into smaller sub-tasks is at the core of many approaches ranging from classic divide-and-conquer algorithms to generating sub-goals in action planning [23]. In RL, temporal abstraction of state sequences converts the problem into a semi-Markov Decision Process (SMDP) [24]. The approach is to abstract the agent's actions using macro-actions (routines), which are composed of primitive actions. This allows for modeling the agent at multiple levels of abstraction. This approach has a parallel with the hierarchical structure of human and animal learning [25]. The concept of hierarchical RL has generated important advancements such as option-learning [26], universal value functions [27], option-critic [28], FeUdal networks [29], data-efficient hierarchical RL known as HIRO [30] and many others. The main advantages of hierarchical RL are transfer learning (re-using previously learned skills and sub-tasks for new objectives), scalability (decomposition of large problems into smaller ones and/or avoiding the curse of dimensionality) and generalization (the combination of smaller sub-tasks to generate new skills and avoiding super-specialization) [31].

In this article, we use a policy selector to select policies fitted to a current environmental sub-space. This approach resembles options learning algorithms [26], and is closely related to prior work on sub-policy selection methods [32] in which sub-policies are hierarchically structured to perform new tasks. Sub-policies are primitives that are pre-trained in a similar environment but with different rewards. Our policy selector (similar to the master policy in [32]) learns to optimize a global reward given a set of *low-level policies*. Low-level policies are specialized, pre-trained policies. However, unlike previous work which focused on meta-learning, our main objective is to learn to dog-fight optimally against different opponents, by dynamically switching between the low-level policies. Because of the complexity of the environment and the task, we do not iterate between the training of the policy selector and the low-level policies, i.e., the low-level policy agents' parameters (weights) are not updated in tandem with training the policy selector.

III. Background

A. Reinforcement Learning

In short, a reinforcement learning (RL) agent is concerned about learning what actions to take to maximize a numerical reward signal given the current state of an environment [33]. Reward signals are provided by the environment and the learning is performed in an iterative way by trial-and-error. Therefore, reinforcement learning is different from both supervised and unsupervised learning. Supervised learning is performed on a training set of labeled examples provided by an external supervisor. Unsupervised learning does not work with labels but instead looks to find some hidden structure in the data [34]. One of the main challenges in

reinforcement learning is to manage the trade-off between exploration and exploitation. As the RL agent interacts with the environment through actions, it starts to learn the choices that eventually return high reward. Naturally, to maximize the reward it receives, the agent should exploit what it has already learned by selecting the actions that resulted in high rewards. However, to discover the optimal actions, the agent has to take the risk and explore new actions that may lead to higher rewards than the current best-valued actions.

B. Maximum Entropy Reinforcement Learning

In RL, knowing the best way to explore while exploiting is non-trivial, environment-dependent, and still an active area of research [35]. Maximum entropy RL theory provides a principled way to address this particular challenge, and has been a key element in many recent RL advancements, providing improved exploration and faster learning [36]–[41]. Given a Markov decision process (MDP) with a set of states S, a set of actions A, a transition function T and a reward function R, forming a tuple $< S, A, T, R >$ [42], a stochastic policy $\pi : S \rightarrow A$ is a mapping from states to probabilities of selecting each possible action, where $\pi(a|s)$ represents the probability of choosing action a given state s. In maximum entropy RL, as in an MDP, the goal is to find the optimal policy π^* that provides the highest expected sum of rewards, while additionally maximizing the entropy of each visited state, leading to the expression [41]:

$$\pi^* = \arg\max_{\pi} \sum_{t} \mathbb{E}_{(s_t,a_t)\sim\rho_\pi}[r_t + \alpha\mathcal{H}(\pi(\ \cdot\ |s_t))], \quad (1)$$

where α is the temperature parameter that controls the stochasticity of the optimal policy, ρ_π is the state-action marginal of the trajectory distribution induced by the policy, r_t is a shorthand for the environmental reward $r(s_t, a_t)$ at time t, and $\mathcal{H}(\pi)$ represents the entropy of the policy, $\mathbb{E}[-log(\pi(a|s))]$. This approach allows a state-wise balance between exploitation and exploration. For states with high reward, a low entropy policy is permitted while, for states with low reward, high entropy policies are preferred, leading to greater exploration. The discount factor γ is omitted in the equation for simplicity since it leads to a more complex expression for the maximum entropy case [36]. But it is required for the convergence of infinite-horizon problems, and it is included in our final algorithm. Maximum entropy RL has conceptual and practical advantages. The entropy term leads the policy to explore more widely, and it also allows near-optimal behaviors. For instance, the agent will prefer two equally attractive actions (higher entropy) than a single action that restricts exploration. In practice, improved exploration and faster learning have been reported in many prior works [37], [38].

C. Soft Actor-Critic

Soft actor-critic (SAC) [39] is one of the most successful maximum entropy RL methods and has become a common baseline algorithm in most popular RL libraries, outperforming state-of-the-art methods [39], [40]. Like the deep deterministic policy gradient (DDPG) approach [43], SAC is a model-free and off-policy method, using a replay buffer, where the policy and value functions are approximated using neural networks. In addition, it incorporates a policy entropy term into the objective function facilitating exploration, similar to soft Q-learning [38]. Similar to trust region policy optimization (TRPO) [44] and proximal policy optimization (PPO) [45], SAC uses a stochastic policy and is known to be more stable than DDPG. In short, SAC combines the best of DDPG (sample efficiency) and TRPO/PPO (stability through stochastic policies). As expressed in Equation 1, the SAC policy/actor is trained with the objective of maximizing the expected cumulative reward and the action entropy at a particular state. The critic is the soft Q-function and, following the Bellman equation, is expressed by: $Q(s_t, a_t) = r_t + \gamma\,\mathbb{E}_{s_{t+1}\sim p}[V(s_{t+1})]$, where p represents the state transition probability, and the soft value function is parameterized by the Q-function: $V(s_{t+1}) = \mathbb{E}_{a_t\sim\pi}[Q(s_{t+1}, a_{t+1}) - \alpha \log \pi(a_{t+1}|s_{t+1})]$. The soft Q-function is trained to minimize the following objective function given by the mean squared error between predicted and observed state-action values:

$$J_Q = \mathbb{E}_{(s_t,a_t)\sim\mathcal{D}}\left[\tfrac{1}{2}\big(Q(s_t, a_t) - \hat{Q}(s_t, a_t)\big)^2\right] \quad (2)$$

where

$$\hat{Q}(s_t, a_t) = r_t + \gamma\,\mathbb{E}_{s_{t+1}\sim p}[\bar{V}(s_{t+1})], \quad (3)$$

$\mathcal{D}$ denotes the replay buffer, and $\bar{V}$ is the target value function [46]. Finally, the policy is updated to minimize the KL-divergence between the policy and the exponentiated state-action value function [40], and can be expressed by:

$$J_\pi = \mathbb{E}_{s_t\sim\mathcal{D}}\left[\mathbb{E}_{a_t\sim\pi}[\alpha \log \pi(a_t|s_t) - Q(s_t, a_t)]\right]. \quad (4)$$

D. Reward Shaping

Similar to many real-world problems, the reward provided by the ADT simulation environment for the dogfighting task is sparse. Success depends on achieving a narrow pointing angle and close proximity to the opponent. No other reward such as flying smoothly or getting close to the opponent is natively provided by the environment. This poses a significant challenge for training the RL agent because it may never receive a reward signal, and hence never learn how to perform the task. Therefore, reward shaping [47], [48] is necessary to provide more feedback during training and exploit implicit domain knowledge to accelerate learning. In spite of recent advances to overcome reward sparsity such as replay buffers [43], universal value function approximators for setting intermediate subgoals [27] and hindsight experience replay for learning from failures [49], it is still difficult to successfully achieve sample efficiency. Hence, learning from sparse rewards continues to be an active area of research [50], [51].

In this work, given the computationally intensive nature of the environment (demanding sample efficiency) and the complexity of the task, we performed reward shaping informed by

domain knowledge in order to compose dense reward signals. This choice provided many advantages: sample efficiency, faster learning and the ability to encode expert knowledge and define low-level agents with specific desired behaviors. In the subsequent sections, we describe each of the shaped rewards in detail.

IV. ADT Simulation Environment

The simulation environment used to model the dogfighting scenario was developed by the Johns Hopkins University Applied Physics Lab (JHU-APL). It is an extension of an open-source RL gym environment[1], which was designed for single aircraft flight and leverages JSBSim, a high-fidelity open-source FDM [52]. JHU-APL extended this work to accommodate multiple F16 aircraft and integrated a more robust visualization engine.

The observation space provided by the environment includes ownship position (local plane coordinates, velocity, and acceleration), attitude (Euler angles, rates, and accelerations), aerodynamics (alpha and beta angles), and other miscellaneous values (fuel load, thrust, control surface deflection, and health). Additionally, the position (local plane coordinates and velocity), and attitude (Euler angles and rates), and health of the opponent are given as well. All state information from the simulation environment is provided without modeled sensor noise.

We experimented with a variety of observation spaces throughout the course of development, including the removal or normalization of default simulation state values as well as the addition of derived states. Derived states included egocentric and allocentric polar coordinates as well as airspeed. Additionally, we also experimented with including state changes (deltas) in the observation spaces of the agents with varying degrees of time between observed state values.

A complete list of the observation space of the simulation as well as the observation space of the agents described in this article can be found in the supplemental material.

Interactions with the environment occur at a rate of 50 times per simulation second. The action space is composed of the main inputs to the F-16's flight control system (aileron, elevator, rudder, and throttle) and is continuous. An agent inflicts damage and reduces the health of its opponent whenever the opponent is within the agent's Weapons Engagement Zone (WEZ).

The WEZ is defined as the locus of points in between 500-3000 ft that also lie within a spherical cone of 2° aperture that extends out of the plane's nose (Figure 1). Though the agent is not truly taking shots on its opponent, throughout this article we use the term "gun snap" to refer to attaining this geometry.

The damage per second incurred from within the WEZ is given by

$$D_{wez} = \begin{cases} \frac{3000-d}{2500} & 500ft \leq d \leq 3000ft \\ 0 & otherwise \end{cases} \quad (5)$$

[1]https://github.com/Gor-Ren/gym-jsbsim

where d is the distance between the two aircraft.

The reward given by the environment is equal to the average amount of total damage an agent has dealt to its opponent and is given by

$$r_t = \mathbb{E}_{t' \in [0,T]}[\sum_{n=0}^{t} D_{wez}(t)] \quad (6)$$

where r_t is the reward at time step t and $T = 300$, the maximum duration of an engagement. The engagement

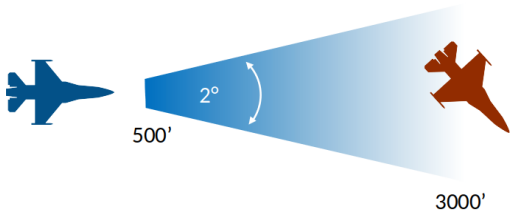


Fig. 1: Weapon Engagement Zone (WEZ).

ends once the simulation time reaches 300 seconds or if either aircraft reaches zero health. An agent "wins" when its opponent's health reaches zero. This can happen if sufficient damage is incurred via gun snaps or the minimum altitude (hard deck) of 1000 feet is breached. The engagement is considered a draw if the simulation times out.

V. Methods

A. Agent Architecture

Our agent, PHANG-MAN (Policy Hierarchy for Adaptive Novel Generation of MANeuvers), is composed of a 2-layer hierarchy of policies. There is an array of policies at the lower level of the hierarchy that have each been trained to excel in a particular region of the state space. A singular policy at the higher level of the hierarchy selects which low-level policy to activate given the current context of the engagement, as shown in Figure 2. We refer to this policy as the policy selector. This architecture allows for the high-level and low-level rewards to be decoupled, and it provides the ability to extend existing hierarchical agents by incorporating additional low-level policies that address specific skill gaps. If the behavior of a low-level policy aids in achieving the policy selector's goal, the policy selector will learn which regions of the state space to employ it.

B. Low-Level Policies

The low-level policies interact with the environment at a frequency of 50 Hz and are trained via Soft Actor-Critic (SAC) [53]. The same observation space is used for all three policies, and a detailed list of the quantities is provided in the supplemental material. Multilayer perceptrons (MLP) with a single hidden layer of 12288 neurons are used for all policies and Q-functions. Wide and shallow MLP networks such as these have been shown to perform better than narrower and deeper MLP networks containing the same number of neurons [54]. What distinguishes the low-level policies

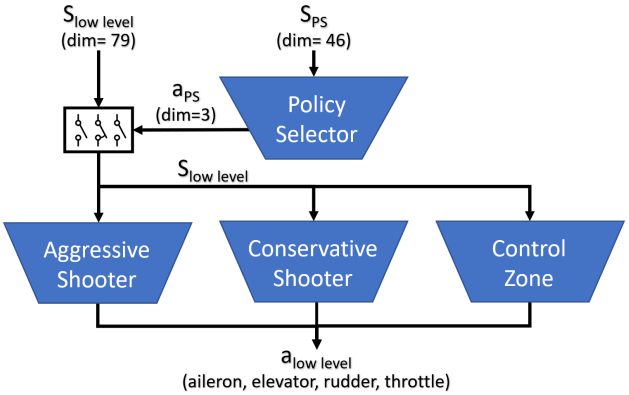


Fig. 2: High-level architecture of PHANG-MAN agent.

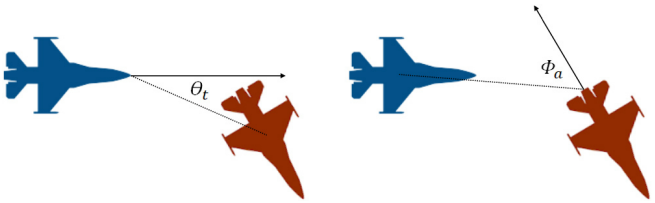


Fig. 3: Depiction of track angle, θ_t, (left) and adverse angle, ϕ_a (right).

are the unique reward functions and the range of initial conditions used during their training. The reward functions are the sum of various independent components that each promote a target behavior.

To facilitate the description of the reward function components, we introduce the following:

- Track angle, $\theta_t \in [0, 180]$, is the angle in degrees between the agent's aircraft nose and the center of the opponent's aircraft, shown in Figure 3 (left). We will use the normalized value $\bar{\theta}_t = \frac{\theta_t}{180}$.
- Adverse angle, $\phi_a \in [0, 180]$ is angle in degrees between the opponent's tail and the center of the agent's aircraft, shown in Figure 3 (right). We will use the normalized value $\bar{\phi}_a = \frac{\phi_a}{180}$.
- The distance between the two aircraft, d, measured in feet.
- Closure rate, v_c, is the rate of change of d and is measured in feet per second.
- The logistic function,

$$S(x, \alpha, x_0) = \frac{1}{1 + e^{-\alpha(x - x_0)}}, \tag{7}$$

parameterized by growth rate, α, and midpoint, x_0.

The equations of each of the reward function components are provided in Table I, and qualitative descriptions are as follows:

- r_{ϕ_a} penalizes the agent for having a non-zero adverse angle.
- r_{θ_t} penalizes the agent for having a non-zero track angle. In Equation 9, λ is unique across policies.
- $r_{relative\ position}$ rewards the agent for attaining the ideal dogfighting geometry in which its track angle and adverse angle are minimized (i.e., pointing directly at its opponent from directly behind its opponent's tail). It also penalizes the agent for allowing its opponent to do the same.
- $r_{closure}$ rewards the agent for getting closer to its opponent from behind and penalizes the agent for allowing its opponent to get closer from behind. It is scaled by distance, in order not to encourage overshooting.
- $r_{gunsnap(blue)}$ rewards the agent when it achieves a minimum track angle and is within a particular range of distances. In Equation 12, $\Gamma_B(d)$ is the range factor that determines the distances at which blue gun snap rewards have the greatest magnitude. It is given by

$$\Gamma_B(d) = \begin{cases} \beta_B(d) \cdot S(d, \frac{1}{50}, 1000) & d < 1950ft \\ \beta_B(d) \cdot [1 - S(d, \frac{1}{50}, 2900)] & d \geq 1950ft \end{cases} \tag{16}$$

where $\beta_B(d)$ is the aggressiveness factor for blue gun snaps. $\beta_B(d)$ is unique across policies and defined later for each instance.

- $r_{gunsnap(red)}$ penalizes the agent when its opponent achieves a minimum track angle and is within a particular range of distances. In Equation 13, $\Gamma_R(d)$ is the range factor for red gun snaps and is given by

$$\Gamma_R(d) = \begin{cases} \beta_R(d) \cdot S(d, \frac{1}{35}, 400) & d < 2250ft \\ \beta_R(d) \cdot [1 - S(d, \frac{1}{200}, 4100)] & d \geq 2250ft \end{cases} \tag{17}$$

$\beta_R(d)$ is the aggressiveness factor for red gun snaps, which is unique across policies and defined later for each instance.

- r_{deck} penalizes the agent for violating a minimum altitude threshold. It is defined by where h is the height above the ground in feet.
- $r_{\ too\ close}$ penalizes the agent for violating a minimum distance threshold when below a maximum adverse angle and is meant to discourage overshooting when pursuing.

TABLE I: Reward function component equations

$$r_{\phi_a} = -\bar{\phi}_a \tag{8}$$

$$r_{\theta_t} = -\bar{\theta}_t^{\lambda} \tag{9}$$

$$r_{rel.\ pos.} = (\bar{\theta}_t - 2) \cdot S(\bar{\phi}_a, 18, \tfrac{1}{2}) - \bar{\theta}_t + 1 \tag{10}$$

$$r_{closure} = \frac{v_c}{500} \cdot [1 - S(\bar{\phi}_a, 18, \tfrac{1}{2})] \cdot S(d, \tfrac{1}{500}, 2900) \tag{11}$$

$$r_{gunsnap(blue)} = \Gamma_B(d) \cdot [1 - S(\bar{\theta}_t, 1e5, \tfrac{1}{180})] \tag{12}$$

$$r_{gunsnap(red)} = -\Gamma_R(d) \cdot S(\bar{\phi}_a, 800, \tfrac{178}{180}) \tag{13}$$

$$r_{deck} = -4 \cdot [1 - S(h, \tfrac{1}{20}, 1300)] \tag{14}$$

$$r_{too\ close} = -2 \cdot [1 - S(\bar{\phi}_a, 18, \tfrac{1}{2})] \cdot S(d, \tfrac{1}{50}, 800) \tag{15}$$

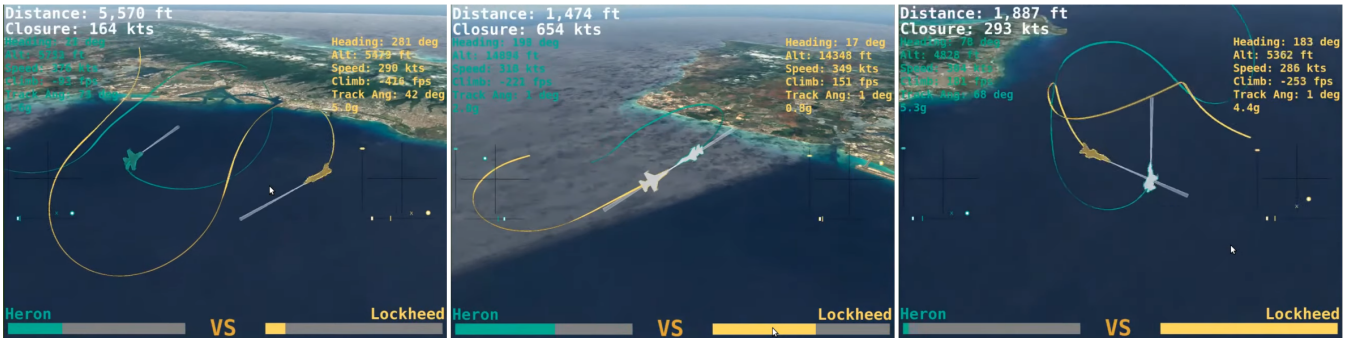


Fig. 4: Typical behavior of the low-level policies driven by reward shaping. Control Zone (left), Aggressive Shooter (middle), Conservative Shooter (right).

A description of each of the low-level policies follows:

Control Zone (CZ): The CZ policy attempts to attain a geometry in which there is nothing the opponent can do to deny the agent's positional advantage. This position is known as the "control zone" in the fighter pilot community and is roughly defined as the cone-like area projecting out of the tail of an aircraft, 20 degrees wide at 2000 feet and 40 degrees wide at 4000 feet [55]. Insights from fighter pilots helped to design the CZ policy's multi-dimensional reward function, which is dependent on track angle, adverse angle, distance to opponent, height above the hard deck, and closure rate. The CZ reward function is equal to the sum of the rewards given by Equations (10)-(15), with $\beta_B(d) = 3$ and $\beta_R(d) = -3$.

The surfaces of select reward function components of the CZ policy are shown in Figure 5. The CZ policy was trained with the widest range of initial conditions of all the low-level policies and included uniformly random positions, Euler angles, and body-frame velocities. Representative behavior of the CZ policy is show in Figure 4 (left).

Aggressive shooter (AS) and Conservative shooter (CS): The most impactful difference between the two shooter policies and the CZ policy is that $r_{\ rel.\ pos.}$ is replaced with r_{θ_t} (9), with $\lambda = \frac{4}{9}$. Since there is no dependence on ϕ_a, the shooter policies learn to attack from all sides, including head on.

The differences between the two shooter policies are relatively subtle. The most notable difference between the two is the AS policy gun snap reward is greater in magnitude at closer distances whereas the CS policies gun snap reward is constant regardless of distance. Qualitatively as a result, the AS policy takes shots that will yield the most damage but potentially leave it susceptible to counter maneuvers, and the CS policy prioritizes maintaining an offensive scoring position, even though the magnitude of the points scored may be low. There is a similar distinction on the defensive side as well. The CS policy avoids gun snaps equally from all distances, making it a relatively aggressive evader, whereas the AS policy avoids gun snaps from close range more than those from farther away, making it a less aggressive evader.

Both the AS and CS policy were trained with initial conditions that place the opponent in the agent's WEZ or vice versa. This increased the ratio of WEZ to non-WEZ memories stored in the replay buffer, biasing the agent's learning toward regions of the state space where offensive and defensive gun snap rewards are the most significant. Additionally, since the time from initial gun snap to episode conclusion was generally observed to be short (sometimes only 1 step), the total health of both aircraft was increased by 10x, to further increase in the ratio of WEZ to non-WEZ memories stored in the replay buffer. The reward function of both the AS and CS are given by

$$r_{AS/CS} = r_{\theta_t} + r_{gunsnap(blue)} + r_{gunsnap(red)} + r_{deck}. \quad (18)$$

Although $r_{AS/CS}$ is the sum of the same components for both the AS and CS policy, the reward components $r_{gunsnap(blue)}$ and $r_{gunsnap(red)}$ themselves are unique. Specifically, for the AS policy,

$$\beta_B(d) = \begin{cases} 2(\frac{2900-d}{1900} + 1) & 1000ft \leq d \leq 2900ft \\ 2 & otherwise \end{cases} \quad (19)$$

and

$$\beta_R(d) = \begin{cases} -2(\frac{4100-d}{3700} + 1) & 400ft \leq d \leq 4100ft \\ -2 & otherwise \end{cases}. \quad (20)$$

For the CS policy $\beta_B(d) = 3$ and $\beta_R(d) = -3$. The surfaces of select reward function components of the AS and CS policies are shown in Figure 6. Representative behaviors of the AS and CS policy are shown in Figure 4 (middle, right).

C. Policy Selector (PS)

At the top of the hierarchy the policy-selector acts like an intelligent de-multiplexer, identifying which of the low-level policies to activate given the current context of the engagement. Our approach is similar to options learning [26], however we do not estimate the terminal conditions directly. Instead, the policy selector is evaluated periodically at a frequency of 10 Hz, which is every fifth action that the low-level policies make. This is much like the Meta Learning Shared Hierarchies architecture [32]. Unlike that approach, we independently train the low-level policies to have target behaviors and then freeze their parameters (weights), which allows training the policy selector without the complications

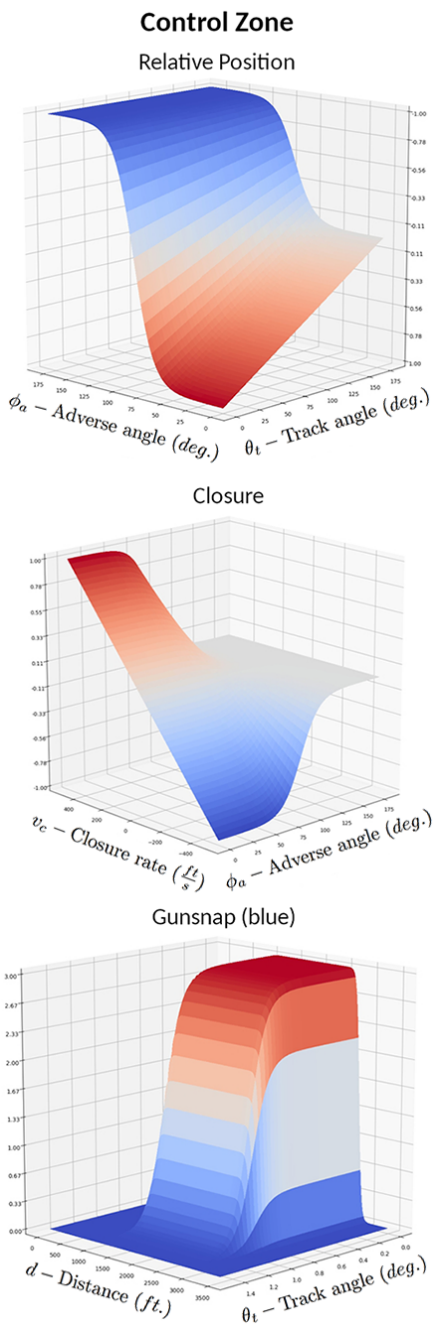


Fig. 5: $r_{\ rel.\ pos.}$, $r_{closure}$, and $r_{gunsnap(blue)}$ components of the CZ policy's reward function.

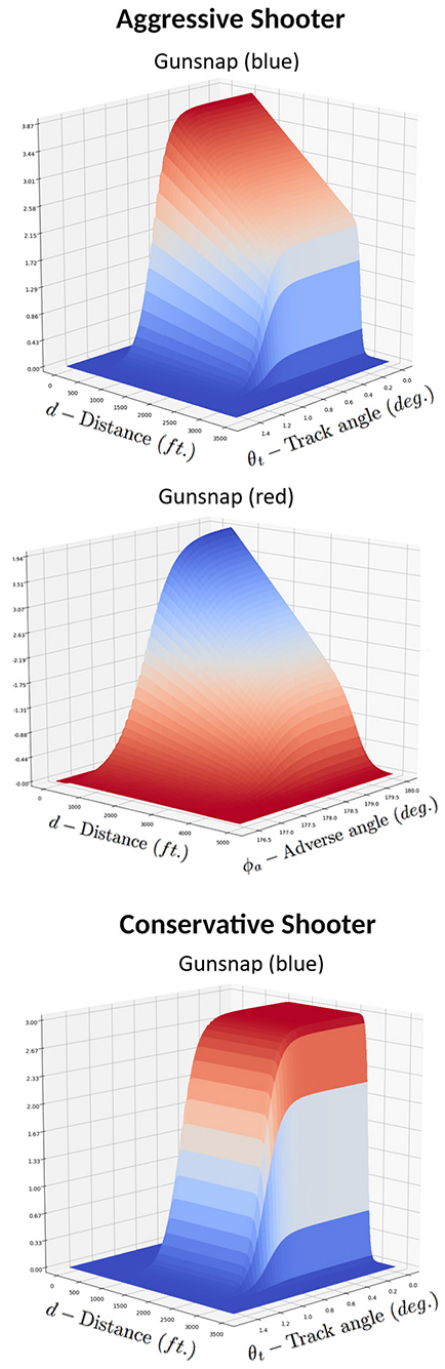


Fig. 6: $r_{gunsnap(blue)}$ and $r_{gunsnap(red)}$ components of the AS policy's reward function (top) and $r_{gunsnap(blue)}$ component of the CS policy's reward function (bottom).

associated with non-stationary policies [30]. This simplifies the learning problem and permits training and re-use of agents in a modular fashion.

The policy selector, which has a discrete action space, is implemented using the same SAC algorithm as the low-level policies. Although an SAC implementation for discrete actions is viable for choosing low-level policies [56], the continuous version of SAC combined with argmax demonstrated better performance. This is unlikely to hold true for larger action spaces, where discrete action space sampling methods such as Gumbel-Softmax are critical to ensure proper exploration [57]. The policy selector's state space is a subset of that used to train the low-level policies, where accelerations, control surface positions, and other quantities are omitted. The full list of values is given in the supplemental material.

The policy selector's reward function is given by,

$$r_{PS} = r_{\theta_t} + r_{gunsnap(blue)} + r_{gunsnap(red)} + r_{deck}, \quad (21)$$

with $\beta_B(d) = 5$, $\beta_R(d) = -5$, and $\lambda = 1$. The dominant terms in r_{PS}, $r_{gunsnap(blue)}$ and $r_{gunsnap(red)}$, resemble the

damage calculation within the environment (5), which is sparse. This ensures that the PS learns strategies that are well aligned with the scoring metric of the environment. To facilitate learning and avoid the challenges associated with sparse rewards, we include a relatively smaller penalty that is directly proportional to track angle. We also include r_{deck} to prevent the PS from learning risky switching combinations that can cause the plane to fly too low, even though each of the low-level policies have been trained to avoid the deck individually.

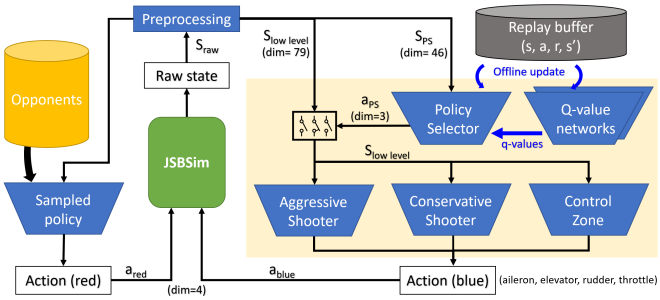


Fig. 7: Training architecture of the PHANG-MAN agent.

In Figure 7, we summarize the hierarchical structure of the agent along with its training framework. Notice that a single-instance version of the offline updating is shown here. The hyperparameters for the low-level policies and policy selector are given in Table II.

TABLE II: List of common hyper-parameters used in the SAC training.

Parameter	**Value**
Optimizer	Adam
Replay buffer size	5.0e6
Number of hidden layers	1
Number of hidden neurons	7168 (Policy Selector)
	12288 (CZ, AS, CS)
Batch size	256
Learning rate	2.0e-4
Discount factor γ	0.99
Soft τ	1.0e-3
Entropy target τ	-3 (Policy Selector)
	-4 (CZ, AS, CS)
Activation function	ReLU
Target update interval	1

VI. Training Framework and Evaluation

In this section, we describe the training and evaluation frameworks as well as additional training strategies used to develop the PHANG-MAN agent.

A. Training Framework and Hardware

For this effort, a custom off-policy distributed training framework was developed in Pytorch (v1.3.1). The framework is similar to the Ape-X architecture [58] but enhanced to accommodate SAC. It is composed of distributed actors, a central learner, and a central prioritized experience replay (PER) buffer. A high-level architecture diagram can be found in supplemental material. Each actor has its own instance of the environment and a copy of the most recent policy weights. The centralized PER buffer managed the most significant experiences, while the central learner continuously updated the policy and value networks. A single training instance on an Amazon EC2 P3.16xlarge (64 CPUs, 8 V100 GPUs, 488 GiB RAM) instance could run 21 actors and was CPU limited due to the computationally demanding JSBSim environment. Training a policy selector network took up to a week, and the low-level policies took up to a month to achieve maximum performance.

B. Training Strategy

In addition to the diverse reward shaping and the distributed training framework, the following components were also critical to train agents and optimize them for the ADT competition:

- Dynamic selection of opponents (curriculum learning): Agents initially trained only against opponents with scripted behaviors that were considered "easy". Once the agent achieved a win/loss ratio greater than 50%, additional "intelligent" opponents were introduced, following a game-theoretic approach [59], [60]. These additional opponents included previous iterations of PHANG-MAN and individual low-level policies. Some of these agents had more defensive or offensive tactics, while others were meant to mimic the behavior of other competitors' agents. After the agent played every opponent 100 times and had an overall win/loss ratio greater than 50%, opponents were sampled from a probability distribution proportional to the win/loss ratio of the last 100 matchups against every opponent. The distribution was dynamically updated after each matchup. This created more matches against challenging opponents and prevented redundant matches against opponents that the agent was able to consistently defeat. The probability of playing any opponent was clipped to a minimum of 0.2% and a maximum of 11.7%. Thirty different opponents were employed during training in addition to self-play.
- Truncation of state and action values: Significantly different outcomes from identical scenarios were observed when deploying on hardware with a different GPU architecture than the training hardware (e.g. Volta vs. Tesla). Sometimes, these differences were large enough to cause a loss that would have otherwise been a win and vice versa. We observed identical inputs to identical networks that were running within identical docker containers producing different outputs, on the order of 1e-6. We speculate that the cause is due to different hardware-based floating point rounding algorithms across architectures, but further investigation is required. Regardless, when

compounding these differences 50 times per simulated second, the F-16 FDM quickly drifts from its rightful state. To synchronize the performance of the agent when deployed on different GPU architectures, we trained all policies with the state values truncated to six decimals and action values truncated to two decimals.

- Initial conditions: Some of the state variables such as relative position, altitude and velocity defined the positional advantage or disadvantage at the beginning of each episode. These initial conditions were classified into categories going from highly defensive to highly offensive positions and were selected according to the policy's specific goals. For example, the AS and CS shooter policies were mostly trained in highly offensive and defensive initial conditions whereas the CZ policy was trained across a more uniform range of initial conditions that balanced mixture included offensive, defensive and neutral.
- Stochastic multi-step returns: For temporal distance learning, multi-step returns often lead to faster learning when a suitably tuned number of steps in the future is used [61]. Instead of tuning a fixed value, we defined the maximum number of steps in the future and uniformly sampled values up to that maximum, storing this value with every experience in the PER buffer. We found 25 steps to be an effective maximum for the low-level policies and 5 for policy selector, which accounts for 0.5 seconds of simulation time in both cases.

C. Performance Metrics during Training and Validation

To closely monitor the agent's learning across multiple opponents throughout training, we computed performance metrics for each individual opponent. In Figure 8 we illustrate the performance against one particular opponent, a finite state machine named Bud-FSM. This graph is the result of an individual training run consisting of tens of thousands of engagements against a wide variety of opponents, in addition to Bud-FSM. Having a diverse set of opponents and the respective performance metrics was important to identify the strengths and weaknesses of the trained agents. These metrics also provided insight on how to effectively group complementary low-level policies together, into what we will refer to as a team. For example, a particular opponent that forced a substantial amount of deck hits indicated the need to include a team-member with better low-altitude tactics.

During the ADT events, competitor's agents were first evaluated according to their performance against the scripted agents developed by JHU-APL. Therefore, to select the best performing agents, we created a benchmark composed of five opponents and played twenty-five matches against each, with a fixed set of initial conditions ranging from defensive to offensive. These initial conditions closely resemble those from previous ADT events and are referred to as ADT initial conditions. The insights gained from these evaluations informed future reward function iterations.

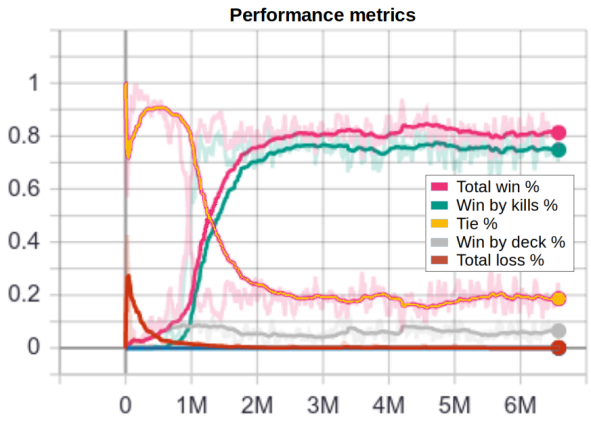


Fig. 8: Example of performance metrics during training against the agent Bud-FSM.

VII. Experimental Results

A. Trained Low-Level Policies

The training of low-level policies was monitored with the same performance metrics (Section VI-C) used to evaluate the hierarchical RL agents. The goal was to train the best performing low-level policy within each category of behavior (CZ, AS, CS) shaped by the reward functions. Their behaviors were qualitatively evaluated by experts (former pilot, Air Force officer) watching a few playbacks, and reward functions were interactively shaped until achieving their final forms (Section V-B). We tested all the low-level policies against scripted ADT agents and previous generations of the PHANG-MAN agent, using a variety of initial conditions. We observed that their performance was sensitive to initial positions and the particular opponent, implying that a Pareto type of dominance was not observed. For example, in head-to-head offensive initial positions, as expected, the AS policy performed better; while in neutral positions, in which positioning was important, the CZ policy performed better.

In Table III, we list all the trained low-level policies. Notice that for training CZ policies, we used 50% random and 50% offensive initial positions, while for shooter policies, we primarily focused on offensive positions. The rationale was to optimize training by narrowing down the possible state-space. Further, in Table V, we show the performance of two low-level policies against several hierarchical policies. The most aggressive (highest number of wins by gun snap) and most defensive (lowest number of Losses) opponents are agents $\{7, 9b\}$ and $\{9b, 11a, 11b\}$, respectively. Interestingly, the CS policy performed better against aggressive than defensive agents with average 28 and 22 wins, respectively; while the AS policy performed slightly better against defensive than offensive agents, with average 22 and 19 wins, respectively.

B. Effectiveness of the Policy Selector

In Figure 9, examples of the policy selector's dynamic behavior are shown as a function of track angle, one of the most telling state variables. In the match against the CS policy (top), the agent exploits the initial positional advantage

(smaller track angle), closes the distance while reducing the track angle, it overshoots once (step 500) but gets persistent gun snaps (steps 1100 to 1300) and ends the engagement victoriously.

Percent utilization per episode of each low-level policy was calculated during policy selector training. When plotted over training episodes, unique usage profiles for different opponents are observed. As shown in Figure 10, the utilization profiles differ significantly when playing against an opponent that makes random actions every step, Randy (top), and an intelligent, aggressive agent such as itself (bottom).

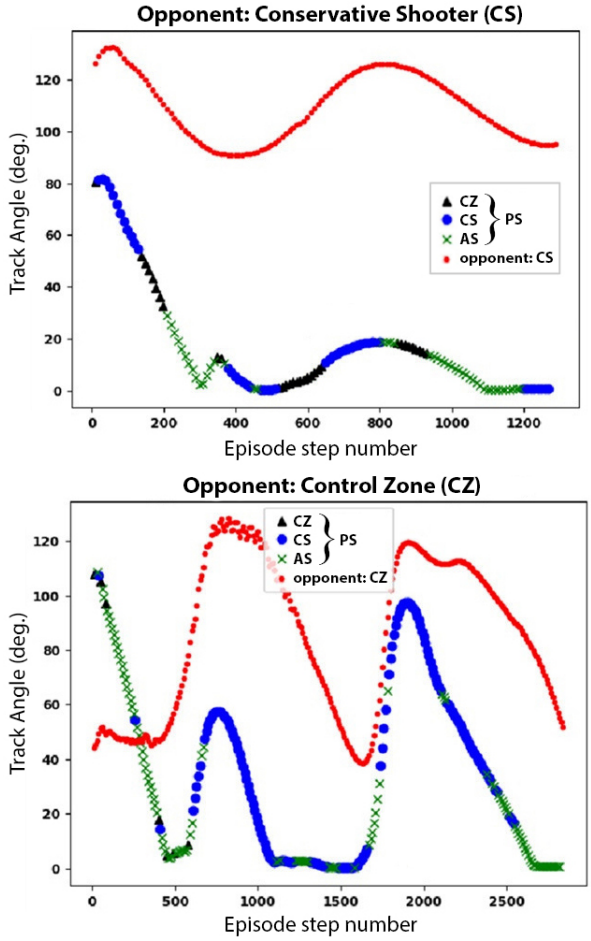


Fig. 9: Low-level policy selection throughout an episode. Policy selector (PS) with a team of 3 low-level policies: CZ, CS, and AS. Opponents: CS (top) and CZ (bottom). Each dot represents a selection made by the PS every 5 steps.

During evaluation, we observed that the win rate of the hierarchical policy selector agent against any individual opponent was at least as great as the win rate of the highest performing low-level policy by itself. Additionally, across many of the opponents, the performance of the policy selector was significantly greater than the highest performing individual low-level policy. This indicates that the policy selector is able to effectively leverage the strengths of the low-level policies as well as generate unique and effective strategies by combining complementary behaviors. The superior performance of hierarchical policy selector agents over single-policy opponents can be seen in Table V of the following subsection.

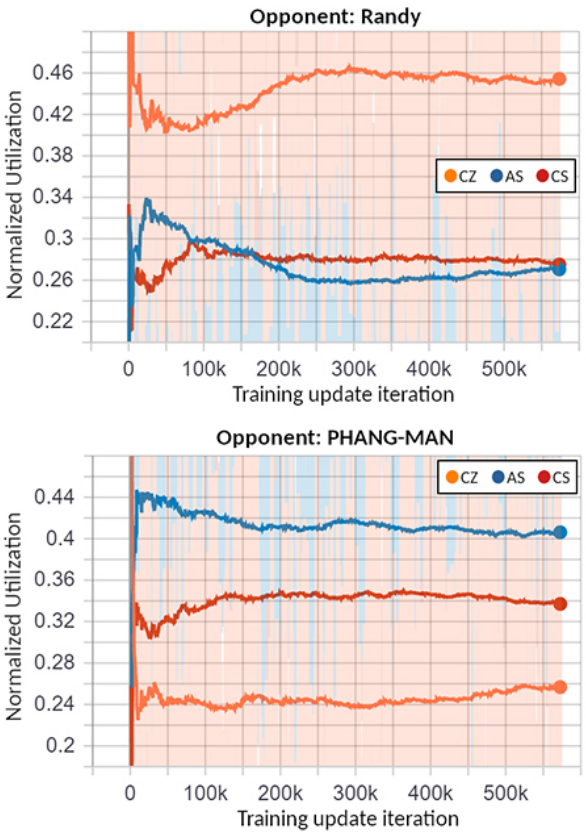


Fig. 10: Normalized utilization of low-level policies vs. PHANG-MAN (self-play) and Randy (random maneuver agent).

C. Validation and Evolution of the Hierarchical Agents

Using the training framework and techniques described in Section VI, we trained and evaluated a variety of agents, and observed a progression of their abilities. In Table III, we list the individual low-level policies used in the iterations of our hierarchical agent and provide a brief overview of each.

In Table IV, all iterations of our hierarchical agent are presented. The table details the reward function used to train the policy selector, the team members (low-level policies), and the initial conditions. These agents follow a chronological order and were composed by two or three of the low-level policies listed in Table III. Each one of the agents was evaluated by playing 100 episodes against three unique opponents. The 100 episodes are the result of 50 unique scenarios played as both blue and red, and they have a near equal mixture of offensive, defensive, and neutral initial conditions, similar to the ones observed during the initial ADT events. The three opponents were chosen for their resemblance to previous iterations of competitors' agents and are a) Agent 1 from Table IV, b) CS single-policy agent and c) AS single-policy agent. A summary of the evaluation

results, with the total delta between wins and losses (Wins-Losses) across opponents, is shown in Table V. Agent 9b was submitted to the ADT final event and was selected based on its superior Wins-Losses delta over the three representative evaluation opponents. Moreover, this agent was among the most aggressive (highest number of wins by gun snap) as well as among the most defensive (lowest number of losses) agents.

Policy	Description
CZ-1	Agent trained for ADT preliminary event with a reward function prioritizing attaining control-zone (Figure 5) and action frequency of 10 Hz. *Reward*: sum of the rewards given by Eqs. (10)-(14). *Initial conditions*: 50% random, 50% WEZ (1/2 offensive), with 25% low-altitude bias.
CZ-2	Similar to CZ-1 agent but trained with action frequency of 50 Hz and additional too close penalty. *Reward*: sum of the rewards given by Eqs. (10)-(15). *Initial conditions*: Same as CZ-1.
CS	Agent trained for ADT preliminary event with a reward function prioritizing conservative gun snaps (Figure 6) and action frequency of 50 Hz. *Reward*: Eq. (18) with $\beta_B(d) = 3$ and $\beta_R(d) = -3$. *Initial conditions*: 100% WEZ (1/2 offensive), with 25% low-altitude bias.
AS	Agent trained with a reward function prioritizing aggressive gun snaps (Figure 6) and action frequency of 50 Hz. *Reward*: Eq. (18) with $\beta_B(d)$ given by Eq. (19) and $\beta_R(d)$ given by Eq. (20). *Initial conditions*: 100% WEZ (1/2 offensive), with 25% low-altitude bias.

TABLE III: List of low-level policies used to compose a team.

Agent	Description
1	*Reward*: $r_{\phi_a} + r_{gunsnap(blue)} + r_{gunsnap(red)} + r_{deck}$ *Initial conditions*: 25% random, 25% ADT, 50% WEZ (1/2 offensive), with 25% low-altitude bias. *Team members*: CZ-1 and CS.
2	Same as Agent 1 except: *Team members*: CZ-2 (1 week of training) and CS.
3	Same as Agent 1 except: Historical states are encoded with a convolutional neural network. *Team members*: CZ-2 (3 weeks of training) and AS.
4	Same as agent 3 except: *Team members*: CZ-2 (3 weeks of training) and CS.
5	Same as agent 2 except: *Team members*: CZ-2 (2 weeks training) and CS.
6	*Reward*: Same as Agent 1. *Initial conditions*: 25% random, 25% ADT (Gaussian noise added to avoid overfitting), 50% WEZ (1/2 offensive), with 25% low-altitude bias *Team members*: CZ-2 (3 weeks of training) and AS.
7	*Reward*: Eq. (21) *Initial conditions*: Same as Agent 6. *Team members*: CZ-2 (3 weeks of training), CS and AS.
8	Same as Agent 7 except: State and action values were truncated for the PS.
9a,b	Same as Agent 8 except: State and action values were truncated for all low-level policies. *Team members*: CZ-2 (3 weeks of training), CS and AS (a: 5 days of training, b: 6 days of training).
10	*Reward*: $r_{gunsnap(blue)} + r_{gunsnap(red)}$ *Initial conditions*: Same as Agent 6. *Team members*: CZ-2 (4 weeks of training), CS and AS.
11a,b	*Reward*: $r_{\theta_t} + r_{gunsnap(blue)} + r_{gunsnap(red)} + r_{deck}$ *Initial conditions*: 10% random, 40% ADT (with Gaussian noise), 50% WEZ (1/2 offensive), with 25% low-altitude bias. *Team members*: Same as Agent 10. a: 3 days of training, b: 4 days of training

TABLE IV: Details of hierarchical agent iterations.

D. PHANG-MAN at the ADT Final Event

Ultimately, the final test of our agent was performed live at the ADT final event which consisted of 3 days of competition. On Day 1, competitors faced agents developed by JHU-APL in collaboration with DARPA. Then on Day 2, all competitors played each other in a best of 20 round robin tournament, with starting conditions ranging from offensive to defensive. On Day 3, the four teams with the highest cumulative score competed in a single elimination championship tournament. In each round of the championship tournament, teams played 20 rounds with neutral starting conditions. These teams also had the opportunity to play against a graduate of the USAF F-16 Weapons Instructor Course in a best of 5 match.

Our agent finished the initial 2 days of the competition in 2^{nd} place and qualified for the championship tournament. On Day 3, PHANG-MAN won its semi-final round and was defeated in the final round by Heron Systems' agent, finishing the competition in 2^{nd} place overall. Finally, our agent emerged victorious (5 W - 0 L) from its match with the USAF Weapons Instructor pilot. The match-ups were characterized by PHANG-MAN taking aggressive shots from head on and the side, while also capitalizing on any mistakes that the human pilots made that gave up their control zone. Overall, PHANG-MAN demonstrated strategic use of aggressive and conservative tactics, reflecting its hierarchical architecture. A full video coverage of the final event can be found online[2], and a more detailed description is provided in supplemental material.

VIII. Discussion

A. Low-level policies

Unlike traditional hierarchical RL architectures like HIRO [30], in which low-level policies execute sub-tasks, in our approach low-level policies were able to execute the entire task (i.e., defeating the adversary). In Frans et al. [32], low-level policies were used to solve parts of the task, while in

[2]https://www.youtube.com/watch?v=NzdhIA2S35w

Agent	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9a**	**9b**	**10**	**11a**	**11b**
Number of Agents	2	2	2	2	2	2	3	3	3	3	3	3	3
Opponent: Agent 1 from Table IV													
Total Wins	58	56	44	57	51	56	54	58	60	**64**	58	59	61
Wins by Gun Snap	55	56	42	53	51	54	51	55	57	**62**	55	53	57
Wins by Deck	3	0	2	4	0	2	3	3	3	2	3	**6**	4
Total Losses	18	18	21	15	16	21	20	20	22	15	15	**13**	14
Losses by Gun Snap	18	18	21	15	16	20	20	20	22	15	15	**13**	14
Losses by Deck	0	0	0	0	0	1	0	0	0	0	0	0	0
Wins-Losses	40	38	23	42	35	35	34	38	38	**49**	43	46	47
Opponent: Conservative Shooter													
Total Wins	49	46	42	53	52	53	60	58	58	**62**	38	59	61
Wins by Gun Snap	45	46	40	50	50	52	**59**	56	55	57	36	54	61
Wins by Deck	4	0	2	3	2	1	1	2	3	**5**	2	**5**	0
Total Losses	42	45	29	40	39	28	29	30	26	26	26	23	**18**
Losses by Gun Snap	42	42	29	39	37	28	28	29	25	26	24	23	**16**
Losses by Deck	0	3	0	1	2	0	1	1	1	0	2	0	2
Wins-Losses	7	1	13	13	13	25	31	28	32	36	12	36	**43**
Opponent: Aggressive Shooter													
Total Wins	50	56	50	49	61	60	**67**	60	**67**	65	54	61	62
Wins by Gun Snap	49	54	44	49	60	59	**67**	59	65	64	53	58	60
Wins by Deck	1	2	**6**	0	1	1	0	1	2	1	1	3	2
Total Losses	45	39	23	37	32	25	21	28	21	**16**	18	24	26
Losses by Gun Snap	40	35	23	36	31	24	21	28	19	**16**	17	24	25
Losses by Deck	5	4	0	1	1	1	0	0	2	0	1	0	1
Wins-Losses	5	17	27	12	29	35	46	32	46	**49**	36	37	36
Total Wins-Losses	52	56	63	67	77	95	111	98	116	**134**	91	119	126

TABLE V: Evaluation of the hierarchical agents listed in Table IV.

our application, low-level policies were selected as a way to optimally adapt to the opponent's maneuvers through the use of different reward functions (Section V-B). Although a more extensive analysis of these policies would be interesting, in this work we focused on the final hierarchical agent and its performance.

B. Policy selector

As described in Section VII-B, the policy selector was able to strategically select low-level policies according to its positional advantage (Figure 9) and opponent type (Figure 10). Similar to previous work in which different skills are selected at different stages of the task [32], [62], our high-level policy selected the low-level policy according to the current context of the engagement.

In this work, we employed a curriculum learning strategy (Section VI-B) which selects adversaries according to their performance metrics (Section VI-C), and achieved progressive improvement as demonstrated in Table V. However, a multi-agent training perspective [18] and a more game theoretic approach for population-level training [60] are promising techniques to be explored.

C. Limitations and Future Work

The one-year timeline of ADT provided many new avenues of future research in this problem space. To this end Lockheed Martin is making significant investments to further research and develop this work. Alongside deeper algorithmic research specific to the air-to-air domain we are adapting our current efforts with the goal of full-scale deployment. Enhancing the simulation environment is the first step in this direction: modification and additions to the observation space (e.g., incorporating domain randomization techniques [63], imperfect information, partial and estimated knowledge) and incorporating different platform aerodynamics models (quad-copter, small fixed-wing unmanned aerial vehicles, F-22, among others). Narrowing the gap between simulation and real-world deployment (sim2real) is critical to rapid-prototyping and validation, as shown by several recent studies [64]–[68]. These enhancements will enable smoother transition as we deploy on sub-scale and eventually full-scale aircraft.

IX. Conclusion

The AlphaDogfight Trials sought to challenge competitors to develop high performing agents able to excel in air-to-air

combat. We developed a hierarchical agent that succeeded in competition, ranking 2^{nd} place overall and defeating a graduate of the USAF Weapons School's F-16 Weapons Instructor Course. LM intends to continue investing in this research and fully explore the potential of these algorithms and their path to deployment. The main conclusions of this work can be summarized as: (i) hierarchical RL, along with modern auxiliary techniques, can be successfully applied to air combat scenarios in high-fidelity environments, producing highly effective maneuvers and tactics; (ii) reward shaping of low-level policies provides a mechanism to incorporate expert pilot knowledge; (iii) training framework design choices play a crucial role in the development of a high performing RL agent.

ACKNOWLEDGMENT

The authors would like to thank DARPA and JHU-APL for being great hosts to the program. The authors would like to thank the members of the research and development team who made impactful contributions to the project: M. Tarascio (former VP of Artificial Intelligence at Lockheed Martin), W. Xie, J. Stanco, B. Liston, J. "Vandal" Garrison, and P. Miskech.

Any opinions, findings, conclusions or recommendations expressed are those of the authors and do not necessarily reflect the views of DARPA, the United States Air Force, the United States Department of Defense, or the United States Government.

REFERENCES

[1] R. Isaacs, "Games of pursuit," RAND Corporation, Santa Monica, California, Tech. Rep., 1951.

[2] M. Molineaux, M. Klenk, and D. Aha, "Goal-driven autonomy in a navy strategy simulation," in *Twenty-Fourth AAAI Conference on Artificial Intelligence*, 2010.

[3] G. H. Burgin and L. Sidor, "Rule-based air combat simulation," Titan Systems Inc La Jolla CA, Tech. Rep., 1988.

[4] J. S. McGrew, J. P. How, B. Williams, and N. Roy, "Air-combat strategy using approximate dynamic programming," *Journal of guidance, control, and dynamics*, vol. 33, no. 5, pp. 1641–1654, 2010.

[5] E. Y. Rodin and S. M. Amin, "Maneuver prediction in air combat via artificial neural networks," *Computers & mathematics with applications*, vol. 24, no. 3, pp. 95–112, 1992.

[6] K. Virtanen, J. Karelahti, and T. Raivio, "Modeling air combat by a moving horizon influence diagram game," *Journal of guidance, control, and dynamics*, vol. 29, no. 5, pp. 1080–1091, 2006.

[7] F. Austin, G. Carbone, M. Falco, H. Hinz, and M. Lewis, "Game theory for automated maneuvering during air-to-air combat," *Journal of Guidance, Control, and Dynamics*, vol. 13, no. 6, pp. 1143–1149, 1990.

[8] J. McManus and K. Goodrich, "Application of artificial intelligence (ai) programming techniques to tactical guidance for fighter aircraft," in *Guidance, Navigation and Control Conference*, 1989, p. 3525.

[9] N. Ernest, D. Carroll, C. Schumacher, M. Clark, K. Cohen, and G. Lee, "Genetic fuzzy based artificial intelligence for unmanned combat aerial vehicle control in simulated air combat missions," *Journal of Defense Management*, vol. 6, no. 1, pp. 2167–0374, 2016.

[10] L. A. Zhang, J. Xu, D. Gold, J. Hagen, A. K. Kochhar, A. J. Lohn, and O. A. Osoba, *Air Dominance Through Machine Learning: A Preliminary Exploration of Artificial Intelligence: Assisted Mission Planning*. Santa Monica, CA: RAND Corporation, 2020.

[11] Y. Chen, J. Zhang, Q. Yang, Y. Zhou, G. Shi, and Y. Wu, "Design and verification of uav maneuver decision simulation system based on deep q-learning network," in *2020 16th International Conference on Control, Automation, Robotics and Vision (ICARCV)*. IEEE, 2020, pp. 817–823.

[12] J. Xu, Q. Guo, L. Xiao, Z. Li, and G. Zhang, "Autonomous decision-making method for combat mission of uav based on deep reinforcement learning," in *2019 IEEE 4th Advanced Information Technology, Electronic and Automation Control Conference (IAEAC)*, vol. 1. IEEE, 2019, pp. 538–544.

[13] Q. Yang, J. Zhang, G. Shi, J. Hu, and Y. Wu, "Maneuver decision of uav in short-range air combat based on deep reinforcement learning," *IEEE Access*, vol. 8, pp. 363–378, 2019.

[14] W. Kong, D. Zhou, Z. Yang, Y. Zhao, and K. Zhang, "Uav autonomous aerial combat maneuver strategy generation with observation error based on state-adversarial deep deterministic policy gradient and inverse reinforcement learning," *Electronics*, vol. 9, no. 7, p. 1121, 2020.

[15] Z. Wang, H. Li, H. Wu, and Z. Wu, "Improving maneuver strategy in air combat by alternate freeze games with a deep reinforcement learning algorithm," *Mathematical Problems in Engineering*, vol. 2020, 2020.

[16] A. P. Pope, J. S. Ide, D. Mićović, H. Diaz, D. Rosenbluth, L. Ritholtz, J. C. Twedt, T. T. Walker, K. Alcedo, and D. Javorsek, "Hierarchical reinforcement learning for air-to-air combat," in *2021 International Conference on Unmanned Aircraft Systems (ICUAS)*, 2021, pp. 275–284.

[17] J. Yoo, D. Kim, and D. H. Shim, "Deep reinforcement learning based autonomous air-to-air combat using target trajectory prediction," in *2021 21st International Conference on Control, Automation and Systems (ICCAS)*, 2021, pp. 2172–2176.

[18] Z. Sun, H. Piao, Z. Yang, Y. Zhao, G. Zhan, D. Zhou, G. Meng, H. Chen, X. Chen, B. Qu, and Y. Lu, "Multi-agent hierarchical policy gradient for air combat tactics emergence via self-play," *Engineering Applications of Artificial Intelligence*, vol. 98, p. 104112, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0952197620303547

[19] T. Vogeltanz, "A survey of free software for the design, analysis, modelling, and simulation of an unmanned aerial vehicle," *Archives of Computational Methods in Engineering*, vol. 23, pp. 449–514, 2016.

[20] X. Chen, Y. Chen, and J. Chase, *Mobile robots-state of the art in land, sea, air, and collaborative missions*, 2009.

[21] P. Dayan and G. E. Hinton, "Feudal reinforcement learning," in *Advances in Neural Information Processing Systems*, S. Hanson, J. Cowan, and C. Giles, Eds., vol. 5. Morgan-Kaufmann, 1993.

[22] A. G. Barto and S. Mahadevan, "Recent advances in hierarchical reinforcement learning," *Discrete Event Dynamic Systems*, vol. 13, no. 1–2, p. 41–77, 2003.

[23] J. Schmidhuber, "Learning to generate subgoals for action sequences," in *IJCNN-91-Seattle International Joint Conference on Neural Networks*, vol. ii, 1991, pp. 453 vol.2–.

[24] R. S. Sutton, D. Precup, and S. Singh, "Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning," *Artificial Intelligence*, vol. 112, no. 1–2, p. 181–211, 1999.

[25] M. M. Botvinick, "Hierarchical reinforcement learning and decision making," *Current Opinion in Neurobiology*, vol. 22, no. 6, pp. 956–962, 2012, decision making.

[26] G. Comanici and D. Precup, "Optimal policy switching algorithms for reinforcement learning," in *Proceedings of the 9th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2010)*, 2010, pp. 709–714.

[27] T. Schaul, D. Horgan, K. Gregor, and D. Silver, "Universal value function approximators," in *Proceedings of the 32nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, F. Bach and D. Blei, Eds., vol. 37. Lille, France: PMLR, 07–09 Jul 2015, pp. 1312–1320.

[28] P.-L. Bacon, J. Harb, and D. Precup, "The option-critic architecture," in *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, ser. AAAI'17. AAAI Press, 2017, p. 1726–1734.

[29] A. S. Vezhnevets, S. Osindero, T. Schaul, N. Heess, M. Jaderberg, D. Silver, and K. Kavukcuoglu, "Feudal networks for hierarchical reinforcement learning," in *Proceedings of the 34th International Conference on Machine Learning - Volume 70*, ser. ICML'17. JMLR.org, 2017, p. 3540–3549.

[30] O. Nachum, S. Gu, H. Lee, and S. Levine, "Data-efficient hierarchical reinforcement learning," *Advances in Neural Information Processing Systems*, pp. 3303–3313, 2018.

[31] Y. Flet-Berliac, "The promise of hierarchical reinforcement learning," *The Gradient*, 2019.

[32] K. Frans, J. Ho, X. Chen, P. Abbeel, and J. Schulman, "Meta learning shared hierarchies," in *6th International Conference on Learning Representations, (ICLR 2018)*. OpenReview.net, 2018. [Online]. Available: https://openreview.net/forum?id=SyX0IeWAW

[33] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: A Bradford Book, 2018.

[34] T. M. Mitchell, *Machine Learning*. New York: McGraw-Hill, 1997.

[35] Z.-W. Hong, T.-Y. Shann, S.-Y. Su, Y.-H. Chang, T.-J. Fu, and C.-Y. Lee, "Diversity-driven exploration strategy for deep reinforcement learning," in *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, Eds., vol. 31. Curran Associates, Inc., 2018.

[36] P. Thomas, "Bias in natural actor-critic algorithms," in *Proceedings of the 31st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, E. P. Xing and T. Jebara, Eds., vol. 32, no. 1. Bejing, China: PMLR, 22–24 Jun 2014, pp. 441–448.

[37] J. Schulman, P. Abbeel, and X. Chen, "Equivalence between policy gradients and soft q-learning," *CoRR*, vol. abs/1704.06440, 2017. [Online]. Available: http://arxiv.org/abs/1704.06440

[38] T. Haarnoja, H. Tang, P. Abbeel, and S. Levine, "Reinforcement learning with deep energy-based policies," in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, D. Precup and Y. W. Teh, Eds., vol. 70. International Convention Centre, Sydney, Australia: PMLR, 06–11 Aug 2017, pp. 1352–1361.

[39] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, J. Dy and A. Krause, Eds., vol. 80. Stockholmsmässan, Stockholm Sweden: PMLR, 10–15 Jul 2018, pp. 1861–1870. [Online]. Available: http://proceedings.mlr.press/v80/haarnoja18b.html

[40] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine, "Soft actor-critic algorithms and applications," *CoRR*, vol. abs/1812.05905, 2018. [Online]. Available: http://arxiv.org/abs/1812.05905

[41] B. D. Ziebart, "Modeling purposeful adaptive behavior with the principle of maximum causal entropy," Ph.D. dissertation, Machine Learning Dpt., Carnegie Mellon University, Pittsburgh, PA, 2010.

[42] M. L. Puterman, *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 1994.

[43] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," in *4th International Conference on Learning Representations, (ICLR 2016)*, Y. Bengio and Y. LeCun, Eds., 2016.

[44] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *Proceedings of the 32nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, F. Bach and D. Blei, Eds., vol. 37. Lille, France: PMLR, 07–09 Jul 2015, pp. 1889–1897.

[45] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *CoRR*, vol. abs/1707.06347, 2017. [Online]. Available: http://arxiv.org/abs/1707.06347

[46] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015. [Online]. Available: http://dx.doi.org/10.1038/nature14236

[47] M. J. Mataric, "Reward functions for accelerated learning," in *ICML*, 1994, pp. 181–189. [Online]. Available: https://doi.org/10.1016/b978-1-55860-335-6.50030-1

[48] A. Y. Ng, D. Harada, and S. J. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proceedings of the Sixteenth International Conference on Machine Learning*, ser. ICML '99. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1999, p. 278–287.

[49] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, O. Pieter Abbeel, and W. Zaremba, "Hindsight experience replay," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper/2017/file/453fadbd8a1a3af50a9df4df899537b5-Paper.pdf

[50] P. Rauber, A. Ummadisingu, F. Mutz, and J. Schmidhuber, "Hindsight policy gradients," in *International Conference on Learning Representations*, 2019. [Online]. Available: https://openreview.net/forum?id=Bkg2viA5FQ

[51] R. Agarwal, C. Liang, D. Schuurmans, and M. Norouzi, "Learning to generalize from sparse and underspecified rewards," in *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97. PMLR, 2019, pp. 130–140.

[52] J. Berndt, "Jsbsim: An open source flight dynamics model in c++," in *Modeling and Simulation Technologies Conference and Exhibit*. American Institute of Aeronautics and Astronautics, 2004.

[53] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proceedings of the 35th International Conference on Machine Learning*. PMLR, 2018, pp. 1861–1870.

[54] K. Ota, D. K. Jha, and A. Kanezaki, "Training larger networks for deep reinforcement learning," 2021, arXiv preprint arXiv:2102.07920v1.

[55] "Flight training instruction for basic fighter maneuvering (bfm) advanced strike/fighter nfo training system," Naval Air Training Command, Corpus Christi, Texas, Tech. Rep., 2018.

[56] P. Christodoulou, "Soft actor-critic for discrete action settings," 2019, arXiv preprint arXiv:1910.07207.

[57] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with gumbel-softmax," *arXiv preprint arXiv:1611.01144*, 2016.

[58] D. Horgan, J. Quan, D. Budden, G. Barth-Maron, M. Hessel, H. van Hasselt, and D. Silver, "Distributed prioritized experience replay," in *International Conference on Learning Representations*, 2018. [Online]. Available: https://openreview.net/forum?id=H1Dy---0Z

[59] K. Tuyls and A. Nowé, "Evolutionary game theory and multi-agent reinforcement learning," *The Knowledge Engineering Review*, vol. 20, no. 1, p. 63–90, 2005.

[60] M. Garnelo, W. M. Czarnecki, S. Liu, D. Tirumala, J. Oh, G. Gidel, H. van Hasselt, and D. Balduzzi, "Pick your battles: Interaction graphs as population-level objectives for strategic diversity," in *AAMAS*, 2021, pp. 1501–1503. [Online]. Available: https://dl.acm.org/doi/10.5555/3463952.3464139

[61] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. The MIT Press, 2018.

[62] Y. Lee, S.-H. Sun, S. Somasundaram, E. S. Hu, and J. J. Lim, "Composing complex skills by learning transition policies," in *Proceedings of International Conference on Learning Representations*, 2019. [Online]. Available: https://openreview.net/forum?id=rygrBhC5tQ

[63] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2017, pp. 23–30.

[64] N. Sünderhauf, O. Brock, W. Scheirer, R. Hadsell, D. Fox, J. Leitner, B. Upcroft, P. Abbeel, W. Burgard, M. Milford, and P. Corke, "The limits and potentials of deep learning for robotics," *The International Journal of Robotics Research*, vol. 37, no. 4-5, pp. 405–420, 2018. [Online]. Available: https://doi.org/10.1177/0278364918770733

[65] W. Zhao, J. P. Queralta, and T. Westerlund, "Sim-to-real transfer in deep reinforcement learning for robotics: a survey," in *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, 2020, pp. 737–744.

[66] O. Nachum, M. Ahn, H. Ponte, S. Gu, and V. Kumar, "Multi-agent manipulation via locomotion using hierarchical sim2real," *CoRR*, vol. abs/1908.05224, 2019. [Online]. Available: http://arxiv.org/abs/1908.05224

[67] J. P.Hanna, S. Desai, H. Karnan, G. Warnell, and P. Stone, "Grounded action transformation for sim-to-real reinforcement learning," *Special Issue on Reinforcement Learning for Real Life, Machine Learning, 2021*, May 2021. [Online]. Available: http://www.cs.utexas.edu/users/ai-lab?MACHINELEARNING21-karnan

[68] M. Ranaweera and Q. H. Mahmoud, "Virtual to real-world transfer learning: A systematic review," *Electronics*, vol. 10, no. 12, 2021. [Online]. Available: https://www.mdpi.com/2079-9292/10/12/1491

[69] J. Ide, P. Shenoy, A. Yu, and C. Li, "Bayesian prediction and evaluation in the anterior cingulate cortex," *The Journal of Neuroscience : the Official Journal of the Society for Neuroscience*, vol. 35, pp. 2039–2047, 2013.

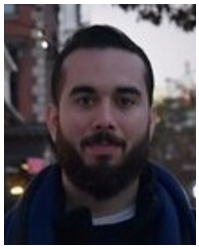
Adrian P. Pope received the B.S. degree in physics from the University of California, Santa Barbara and the M.S. degree in electrical engineering from San Jose State University. He is the co-founder of Primordial Labs, a company focused on developing human-centric autonomy. As a researcher engineer at Lockheed Martin from 2015 to 2021, he served as the principal investigator on several Science and Technology programs. His current work is focused on autonomous control of unmanned systems and human-machine teaming.

Jaime S. Ide received the B.S. degree in mechatronic engineering, and the M.S. and the Ph.D. degrees in engineering (artificial intelligence) from the University of Sao Paulo, Brazil. From 2007-2010, he worked on computational and Bayesian methods applied to neuroimaging at the University of Pennsylvania and Yale University. From 2010 to 2015, he was an Assistant Professor at the Federal University of Sao Paulo, Brazil. From 2016-2019, he worked as an Associate Research Scientist at Yale University School of Medicine. Since 2019, he has been a Research Engineer with the Lockheed Martin Artificial Intelligence Center, where he develops machine learning in unmanned aircraft and cognitive systems. Dr. Ide was the recipient of the Sao Paulo Research Foundation FAPESP's Young Investigators Grants in 2011.

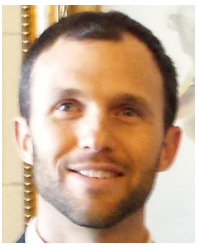
Daria Mićović received the B.S. degree in mathematics from the University of Washington. She worked as a Machine Learning Engineer at the Lockheed Martin Artificial Intelligence Center.

Henry Diaz received the B.S. degree in computer engineering from Florida International University. He is an associate LM Fellow within the Lockheed Martin Artificial Intelligence Center. His work focuses on rapid prototyping and the integration and deployment of machine learning algorithms on size, weight, and power constrained systems.

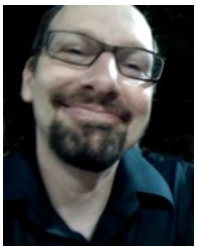
Jason C. Twedt received the B.S. degree in physics from Mansfield University of Pennsylvania and the M.S. from the Virginia Polytechnic Institute and State University in aerospace engineering. Currently, he is the manager of the Computer Vision Systems group at the Lockheed Martin Artificial Intelligence Center, where he leads a team focused on building capabilities that leverage advancements in deep learning and computer vision to create advanced perception capabilities for one or more sensing agents.



Kevin Alcedo received both the B.S. degree in Mechanical Engineering and Applied Mechanics and the M.S. degree in Robotics from the University of Pennsylvania. Currently, he is an Engineering Research Manager at the Lockheed Martin Artificial Intelligence Center where he leads a team focused on the R&D of intelligent agent capabilities. Previously, he was a Machine Learning Engineer at NBC Universal where he developed measurement and optimization algorithms used across the company's entertainment portfolio. He holds various patents in the field of "Soft Robotics".



Thayne T. Walker received the B.S. degree in computer science from Southern Utah University, M.S. in computer science from Colorado State University and the Ph.D. in artificial intelligence at University of Denver. He has background in electronic warfare, multi-source data fusion, cloud computing systems, combinatoric optimization and multi-agent systems. He leads several research and development projects at Lockheed Martin and has published novel algorithms for multi-agent pathfinding.



David Rosenbluth received the B.S. in mathematics and theoretical computer science from Columbia University and the Ph.D. in computation and neural systems from Caltech. He is a LM Fellow within the Lockheed Martin Artificial Intelligence Center and has 25 years of experience in this domain. The current focus of his research is machine learning generative models, neuromorphic systems, and integration of machine learning and machine reasoning. He has published research on the topics of electrophysiology and visual neuroscience, neuromorphic photonics, machine learning, and mathematics.

Lee Ritholtz received the B.S. and M.S. degrees in electrical engineering from the State University of New York at Binghamton. He is the co-founder of Primordial Labs, a company focused on developing human-centric autonomy. He was previously the Director and Chief Architect for the Applied AI organization at the Lockheed Martin Artificial Intelligence Center. His current work focuses on machine learning for autonomous decision making, sensor processing at the edge, and human-machine interfaces for unmanned systems.

Daniel Javorsek II holds several advanced degrees including the Ph.D. in physics from Purdue University. He is a former Program Manager for DARPA and is currently the Commander of Detachment 6, Air Force Operational Test and Evaluation Center (AFOTEC), Nellis Air Force Base, and Director, F-35 U.S. Operational Test Team. As the Commander of AFOTECs Detachment 6 he leads the planning, execution, and reporting on realistic, objective, and impartial operational test and evaluation of fighter aircraft.

SUPPLEMENTARY MATERIAL

Hierarchical Reinforcement Learning for Air Combat at DARPA's AlphaDogfight Trials

Adrian P. Pope, Jaime S. Ide, Daria Mićović, Henry Diaz, Jason C. Twedt, Kevin Alcedo,
Thayne T. Walker, David Rosenbluth, Lee Ritholtz, and Daniel Javorsek II

S1. PHANG-MAN AT THE ADT FINAL EVENT

The ADT final event consisted of 3 days of competition. On Day 1, competitors faced agents developed by JHU-APL in collaboration with DARPA. Then on Day 2, all competitors played each other in a best of 20 round robin tournament, with starting conditions ranging from offensive to defensive. On Day 3, the four teams with the highest cumulative score competed in a single elimination championship tournament. In each round of the championship tournament, teams played 20 rounds with neutral starting conditions. These teams also had the opportunity to play against a graduate of the USAF F-16 Weapons Instructor Course in a best of 5 match.

Our agent, the submission from Lockheed Martin, finished the initial 2 days of the competition in 2^{nd} place and qualified for the championship tournament. On Day 3, PHANG-MAN won its semi-final round and was defeated in the final round by Heron Systems' agent, finishing the competition in 2^{nd} place overall. The tournament bracket and results are shown in Figure S1. The semi-finals and finals can be viewed online.[3]

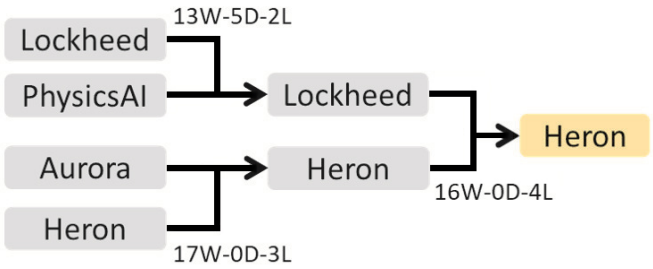


Fig. S1: Day 3 tournament results.

In the championship round, we faced a very aggressive and highly accurate adversary. In most cases, PHANG-MAN did not survive the initial exchange of nose-to-nose gun snaps and matches would end with the opponent having a small amount of health remaining. Although we scored 7% more total shots against our opponent, our average gun snap was from a greater distance and as a result the average damage per gun snap was lower. Our agent appeared to disengage its offense inside of 800 feet in favor of better positioning for the next exchange, while the opposing agent would continue to aggressively pursue head on. Whenever PHANG-MAN survived the initial exchange, it was generally able to achieve and maintain an advantageous position.

We suspect that this bias toward future positioning over immediate scoring is the result of a strategy we used during training in which we increased the health of the agents by a factor of ten. Additionally, by not providing the opponent's health as a state value or a reward proportional to the opponent's health, we inadvertently trained our agent to allow giving away near victories.

Fig. S2: VR enabled cockpit for human vs. AI matchups.

Overall, PHANG-MAN demonstrated strategic use of aggressive and conservative tactics. We believe it has the potential to be a trustworthy co-pilot or wingman that can effectively execute real-world tactics in situations where self-preservation is paramount and aggressive nose-to-nose are not feasible.

For the matches between the top competitors and the human USAF pilot, a high-fidelity VR enabled cockpit was provided by the DARPA and JHU-APL team (Figure S2). This allowed the human pilot to visually track the opponent as he typically

[3]https://www.youtube.com/watch?v=NzdhIA2S35w

would in a real engagement. A dashboard was displayed for the pilot providing a simplified view of pertinent information (e.g track angle, relative distance to opponent, altitude, fuel, etc). In order to provide an additional visual assist for the pilot, an icon pointed out the direction to the opponent when out of view and a red flashing overlay was shown when receiving a gun snap.

PHANG-MAN emerged victorious (5 W - 0 L) from its match with the USAF Weapons Instructor pilot. The match-ups were characterized by PHANG-MAN taking aggressive shots from head on and the side, while also capitalizing on any mistakes that the human pilot made that gave up their control zone.

S2. Training Framework

For this effort, a custom off-policy distributed training framework was developed in Pytorch (v1.3.1). The framework is similar to the Ape-X architecture but enhanced to accommodate SAC. It is composed of distributed actors, a central learner, and a central prioritized experience replay (PER) buffer. Each actor has its own instance of the environment and a copy of the most recent policy weights. The centralized PER buffer managed the most significant experiences, while the central learner continuously updated the policy and value networks.

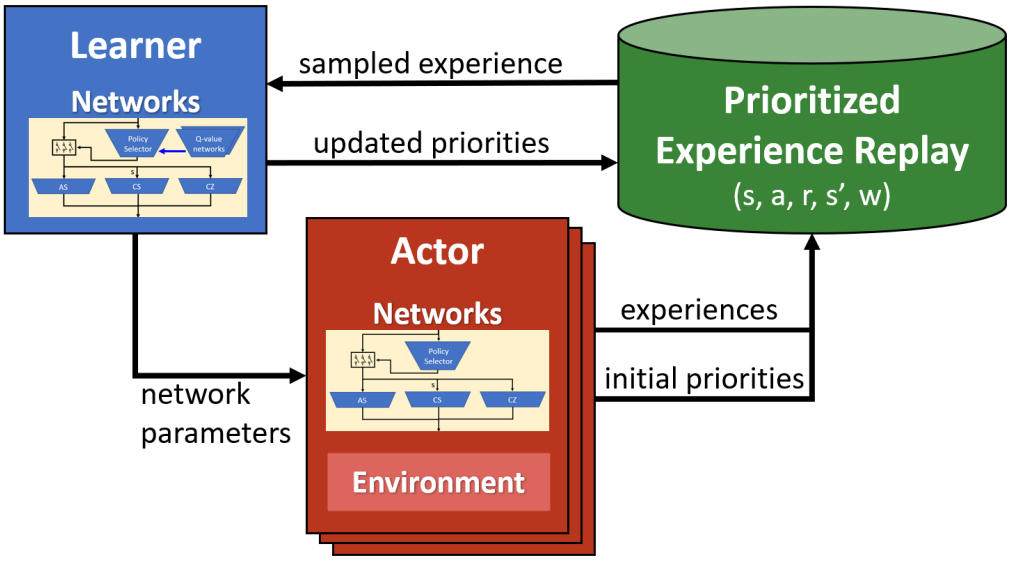


Fig. S3: High-level architecture diagram of the distributed training framework.

S3. STATE SPACE

State	Own	Opp	Delta	Delta Opp
altitude (ft)	LLP / PS	LLP / PS	LLP / PS	LLP / PS
roll (rad)	LLP / PS	LLP / PS		
pitch (rad)	LLP / PS	LLP / PS		
heading (deg)	LLP / PS	LLP / PS		
bodyframe x-axis velocity (ft/s)	LLP / PS	LLP / PS		
bodyframe y-axis velocity (ft/s)	LLP / PS	LLP / PS		
bodyframe z-axis velocity (ft/s)	LLP / PS	LLP / PS		
roll rate (rad/s)	LLP / PS	LLP / PS		
pitch rate (rad/s)	LLP / PS	LLP / PS		
yaw rate (rad/s)	LLP / PS	LLP / PS		
bodyframe x-axis acceleration (ft/s^2)	LLP			
bodyframe y-axis acceleration (ft/s^2)	LLP			
bodyframe z-axis acceleration (ft/s^2)	LLP			
roll acceleration (rad/s^2)	LLP			
pitch acceleration (rad/s^2)	LLP			
yaw acceleration (rad/s^2)	LLP			
alpha: angle of attack (deg)	LLP	LLP	LLP	LLP
beta: sideslip (deg)	LLP	LLP	LLP	LLP
normalized left aileron position	LLP		LLP	
normalized right aileron position				
normalized left flaperon position				
normalized right flaperon position				
normalized speedbrake position				
normalized trailing edge flap position	LLP		LLP	
normalized leading edge flap position	LLP		LLP	
left differential horizontal tail (rad)	LLP		LLP	
right differential horizontal tail (rad)	LLP		LLP	
normalized rudder position	LLP		LLP	
normalized throttle position	LLP		LLP	
fuel in tanks (lbs)	LLP			
simulation time (s)				
engine thrust (lbs)	LLP		LLP	
x position (E) relative to scenario center (ft)				
y position (N) relative to scenario center (ft)				
z position (U) relative to scenario center (ft)				
distance to opponent (ft)	LLP / PS		LLP / PS	
track angle (rad)	LLP / PS	LLP / PS	LLP	LLP
height above altitude hard deck (ft)				
aircraft health				
whether opponent is in the WEZ				
*airspeed (ft/s)	LLP / PS	LLP / PS	LLP / PS	LLP / PS
*bearing to opponent, accounting for roll/pitch (deg)	LLP / PS	LLP / PS	LLP / PS	LLP / PS
*elevation to opponent, accounting for roll/pitch (deg)	LLP / PS	LLP / PS	LLP / PS	LLP / PS
*bearing to opponent, plane as point mass (deg)	LLP / PS	LLP / PS	LLP / PS	LLP / PS
*elevation to opponent, plane as point mass (deg)	LLP / PS	LLP / PS	LLP / PS	LLP / PS

TABLE S1: State space of the environment and quantities used to train the low-level policies (LLP) and policy selector (PS). The Own column includes values of agent itself, the Opp column includes values of the agent's opponent, and the Delta / Delta Opp columns include the change in Own / Opp values over the previous 10 steps. States with an asterisk are derived from the states provided by the simulation environment.